

JOR-V3.1 Collision Risk Module

STANDARD OPERATING PROCEDURE & USER MANUAL

1. ENVIRONMENT & ARCHITECTURE OVERVIEW

The Collision Risk Module operates within a completely self-contained folder structure to prevent analytical cross-contamination. Ensure your local environment contains the following system assets before starting execution:

- **jor_fusion.py** - Interactive terminal scoring interface and baseline configuration template. Holds global calibration constants.
- **jor_pymc.py** - Highly optimized, vectorized PyMC sampling engine handling TruncatedNormal evidentiary probability distributions.
- **jor_pymc_runner.py** - The bridge automation script that runs the MCMC engine, calculates risk multipliers, and performs the safety audit.
- **Safety_Command_Center.py** - Primary Operational Safety Management System (SMS) console. (Located in the root directory)
- **pages/Analytical_Deep_Dives.py** - Post-event statistical validation, infographic reports, and threat-tier separation auditing module.
- **jor_scores.csv** - Automatically generated/appended by the system backend after a tracking profile data session is saved.

2. STEP-BY-STEP OPERATIONAL WORKFLOW

Step 1: Initial Case Ingestion & Scoring Calibration

Before any data can be audited, a case profile must be captured. Because the tracking tools reside within the module subfolder, you must first target the active workspace directory:

1. Open a terminal process and enter the directory:

```
cd Collision_Risk
```

2. Initialize the terminal-based rubric tool to calibrate the profile boundaries:

```
python jor_fusion.py
```

3. **Configuration Verification:** The terminal renders a snapshot of current file parameters (PRIOR_NH, weights). Review and verify compliance.
4. **Data Input Matrix:** Progress through sequential rubric menus to assign scores for:
 - Witness Credibility (C)
 - Environmental / Observation Conditions (E)
 - Physical / Sensor Evidence (P)
 - Flight Behavior Classification (None, Minor, Moderate, or Major Anomaly)

5. Input 'y' when prompted to finalize and save the data tracking session to disk. (If **jor_scores.csv** is missing, the ledger is generated automatically upon closing, injecting structured headers and enforcing clean `.round(3)` decimal limits to prevent floating-point tail leakage across rows).

Step 2: Execute MCMC Processing Engine & Risk Audit

Transform raw categorical scoring values into rigorous, sampled posterior parameter boundaries using the processing automation script.

1. Execute the operational script bridge directly from your terminal session:

```
python jor_pymc_runner.py
```

2. The sampler initializes 4 parallel Markov Chain Monte Carlo (MCMC) tracking chains to resolve evidentiary distributions, compiling 1,000 baseline tracking draws per chain.
3. **Automated Index Mapping:** The bridge script pulls the probabilistic mean output (SOP_Mean) and executes the core safety risk calculation:

```
CRI = SOP_Mean × (1 + Flight_Mod)
```

4. The system executes a discrete classification sweep, indexing tracks automatically into designated Low, Elevated, or Critical hazard threat tiers, and appends records cleanly to disk.

Step 3: Deploy the Aerospace Safety Consoles (Final Review)

Once the underlying case ledger has been completely updated and written to disk, launch the visual web interfaces to audit the finalized tracks:

1. Initialize the primary web-based application interface:

```
streamlit run Safety_Command_Center.py
```

2. The interface generates a secure local server access point (default: **http://localhost:8501**). Open your browser to interact with data assets.
3. **Native Sidebar Navigation:** Streamlit automatically parses the 'pages/' directory, allowing you to switch seamlessly between the main "Safety Command Center" and the "Analytical Deep Dives" interface using the sidebar menu.

3. CORE ANALYTICAL FRAMEWORKS

The user interface environment separates high-level fleet triage from detailed statistical audit suites across two distinct viewports:

A. Primary Command Center (Safety_Command_Center.py)

- **Data Mode Warning:** Permanently flagged in a static "HISTORICAL ARCHIVE" Data Mode to explicitly communicate the static-read nature of the system layout.
- **Kinetic Hazard Frontier:** Maps spatial tracking physics by indexing the probabilistic SOP_Mean against the mathematical kinematic scalar multiplier (Flight_Mod). This viewport relies on layered, background logistic contour gradients to classify emerging flight risk thresholds. Scatter node dimensions dynamically expand

relative to the statistical uncertainty range (95% Credible Intervals) calculated by the backend MCMC sampling matrix.

- **Threat Intelligence Quadrant Map:** A coordinate plotting tool aligning base physicality confidence levels against absolute calculated CRI safety scores. Visually isolates ambiguous observations from solid, high-risk targets. Features a visible, horizontal indicator marking the Critical Action Limit (0.75) boundary line to guide safety protocols.
- **Evidentiary Uncertainty Forest:** An error-bar plot highlighting the precise dispersion limits between CI_2.5% and CI_97.5% across distributions. This enables personnel to evaluate whether a critical warning indicator is driven by stable telemetry or massive-variance data anomalies.

B. Analytics Console (pages/Analytical_Deep_Dives.py)

- **Master Infographic Pipeline:** Provides horizontal hazard potential rankings alongside a monospaced text summary block reporting data-wide metrics, parameters, and tracking averages.
- **T-Tier Buffer Separation:** Enables users to define lower and upper safety buffers. It executes an unequal variance Welch's t-test verification on the fly between threat tiers, reporting the precise t-statistic and p-value to statistically confirm clear threat-tier separation.

4. MANAGING CALIBRATION & SENSITIVITY ANALYSES

To calibrate uncertainty tolerances or adjust weights to align with changing intelligence validation baselines:

CRITICAL CONFIGURATION PROTOCOL: Global constants controlling underlying weights and limits are managed strictly via code adjustments at the head section of `jor_fusion.py`.

Option 1: Reproducible Calibration and MCMC Sensitivity Sweeps (Recommended)

1. Open `jor_fusion.py` using an external configuration or code editor.
2. Edit the designated parameters located below the explicit # Configuration header boundaries:

```
• PRIOR_NH      = 0.20    # Target prior probability
• CALIBRATION_K = 0.20    # Calibration coefficient scalar
• WEIGHT_C      = 0.40    # Witness weight modifier
• WEIGHT_E      = 0.30    # Environmental conditions weight modifier
• WEIGHT_P      = 0.30    # Physical evidence weight modifier
```

3. Commit structural updates to file disk.
4. Run `python jor_pymc_runner.py` inside a completely clean, fresh process environment. The automation bridge loads updated variables directly from code structure, forcing the MCMC sampler to track modifications perfectly.
5. Ensure baseline defaults are restored manually once sensitivity verification sweeps conclude.

Option 2: Memory-Transient Standalone Diagnostics

1. Input 'y' at the terminal challenge prompt to alter constants interactively during runtime inside `python jor_fusion.py`.
2. **Operational Risk:** Variable updates exist exclusively within the context memory buffer of that temporary shell environment. Changes will not pass through when executing `jor_pymc_runner.py`, as the sampler script reads fresh parameters from file arrays during startup.

Data Hot-Reloading: To pull these data updates or new track profiles into your active browser view instantly without restarting the local web server, click the "  Hot-Reload Core Dataset" button on the dashboard sidebar. This explicitly purges Streamlit's internal memory cache and re-reads the underlying CSV ledger fresh.